

Structure



Random Pattern



Permutation Test

11/13/2025 (Week 12)

Jingjing Yang, PhD

Associate Professor of Human Genetics

Jingjing.yang@emory.edu

Hypothesis Test

- Testing **Null (H_0)** vs. **Alternative (H_a)** hypothesis
- Choose an appropriate **Test Statistic**
- **P-value:** The probability of getting a test statistic as or more extreme than the one from your current data, when the null hypothesis is true.
- **Option 1:** Calculate from the known test statistic distribution (cdf/pdf) under the null hypothesis, e.g., Standard Normal distribution $N(0, 1)$, Student's t distribution, F distribution, Chi-square distribution
 - For example, two-sample t-test statistic
$$T = \frac{\widehat{\mu}_1 - \widehat{\mu}_2}{s/\sqrt{n}}$$
 - Testing if two groups of data have the same mean
$$H_0: \mu_1 = \mu_2 \text{ vs. } H_a: \mu_1 \neq \mu_2$$
- **What is the assumed data normality is violated?**

Non-parametric Wilcoxon Rank-Sum Test

- Wilcoxon signed-rank test or Wilcoxon rank-sum test:
`wilcox.test()`, alternative to t-test without assuming normality
 - Test if the median of one group differs from xx: `wilcox.test(x, mu=xx)`
 - Test if medians of two groups differ: `wilcox.test(x, y)`
 - Test if the median difference between paired samples is 0:
`wilcox.test(before, after, paired = TRUE)`
- Assumptions
 - Independent samples
 - Measurement scale can be ranked
 - Distributions of the two groups have similar shapes

Permutation Test: Non-parametric

- **Permutation test:** obtaining p-value by permuting the current observed data to simulate null scenarios
- A permutation test gives a simple way to compute the sampling distribution for any test statistic, **under the null hypothesis**
 - No assumption of the normal or any other distribution for the test statistic under the null hypothesis
 - Suitable for small sample sizes

Permutation Test Procedure

- Generate a large number of data sets under the null hypothesis by **permuting the observed data/outcome**
 - Permute the dependent variable (or outcome) such as group labels (e.g., for testing mean difference between two groups) or measurements, and calculate test statistic values with permuted data
 - Permuting the dependent variable (or outcome) will generate a new data under the null hypothesis (e.g., no association)
 - Permuting the dependent variable (or outcome) will break the association between the dependent variable (or outcome) and the test covariate if null is false
- Calculate test statistic value from the real observed data
- The ranking of the real test statistic value among the shuffled test statistic values gives a **permutation test p-value**

Visual explanation for Permutation Test

- <https://www.jwilber.me/permutationtest/>

Example 1: test two-group mean difference

- Simulate 3000 samples with 1500 not carrying a mutation and 1500 carrying a mutation.
- A null phenotype ``null.y`` and an alternative phenotype ``alt.y`` vectors were simulated from ``N(0, 1)`` and ``N(carrier/2, 1)``, respectively.

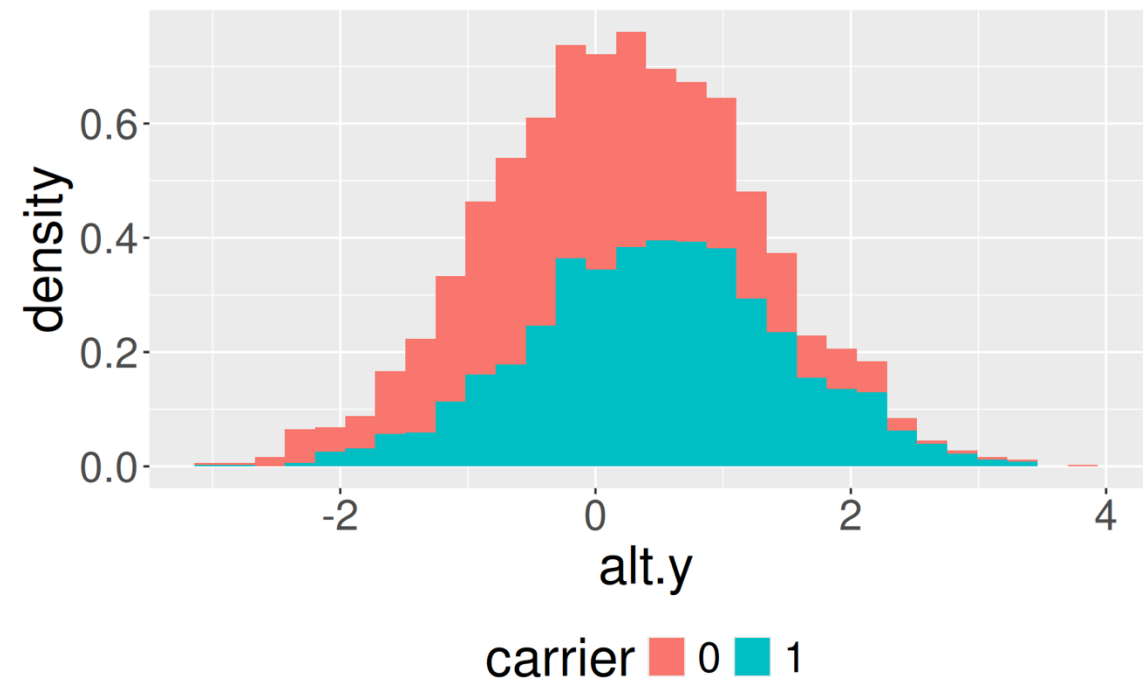
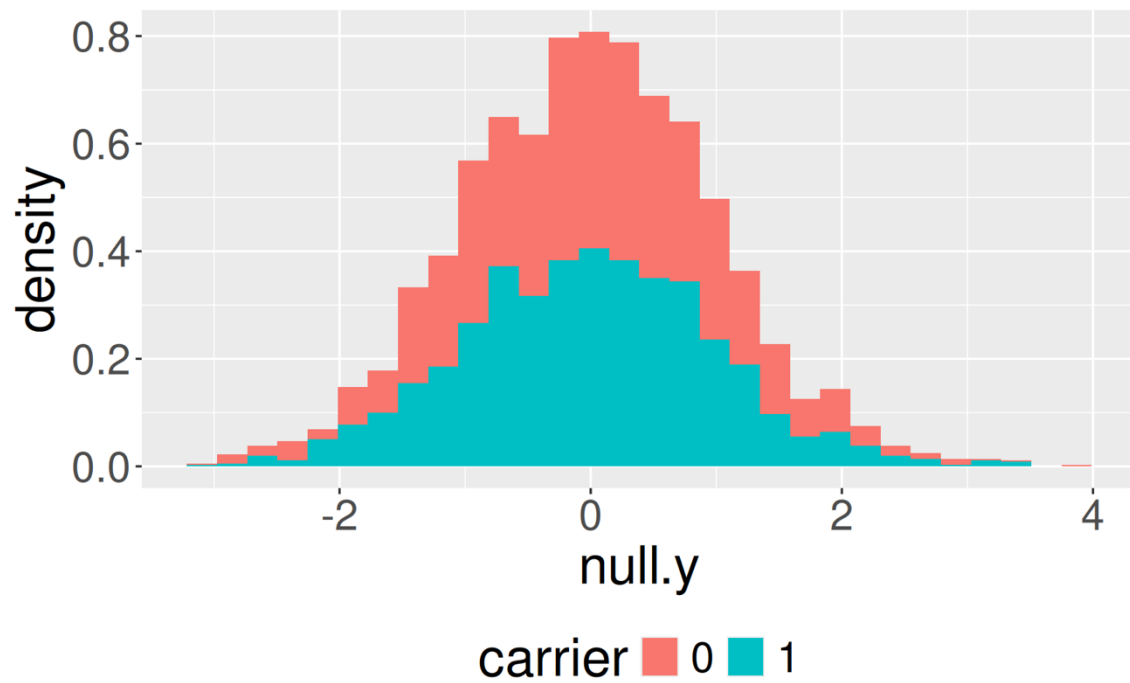
```
set.seed(2025)

## Generate carrier label: 0, 1
carrier <- rep(c(0,1), c(1500,1500))
## Generate null.y under null distribution
null.y <- rnorm(3000)
## Generate alt.y under alternative distribution
alt.y <- rnorm(3000, mean = carrier/2)

## Create data frame for plot
null_dt <- data.frame(carrier = factor(carrier), null.y = null.y)
## Plot null.y data per carrier group
ggplot(null_dt, aes(x = null.y, fill = carrier)) +
  geom_histogram(aes(y = stat(density) ))

## Create data frame for plot
alt_dt <- data.frame(carrier = factor(carrier), alt.y = alt.y)
## Plot alt.y data per carrier group
ggplot(alt_dt, aes(x = alt.y, fill = carrier)) +
  geom_histogram(aes(y = stat(density) ))
```

Phenotype Vectors under Null and Alternative Distributions



Standard two sample t test results

```
```{r}  
t-test with phenotype generated under the null distribution
t.test(null.y ~ carrier, var.equal = TRUE)
```
```

Two Sample t-test

```
data: null.y by carrier  
t = 0.32582, df = 2998, p-value = 0.7446  
alternative hypothesis: true difference in means between group 0 and  
group 1 is not equal to 0  
95 percent confidence interval:  
 -0.06007461  0.08401862  
sample estimates:  
mean in group 0 mean in group 1  
 -0.005507201    -0.017479206
```

Standard two sample t test results

```
```{r}  
t-test with phenotype generated under the alternative distribution
t.test(alt.y ~ carrier, var.equal = TRUE)
```
```

Two Sample t-test

```
data: alt.y by carrier  
t = -13.534, df = 2998, p-value < 2.2e-16  
alternative hypothesis: true difference in means between group 0 and  
group 1 is not equal to 0  
95 percent confidence interval:  
 -0.5702612 -0.4259349  
sample estimates:  
mean in group 0 mean in group 1  
 -0.0235943      0.4745038
```

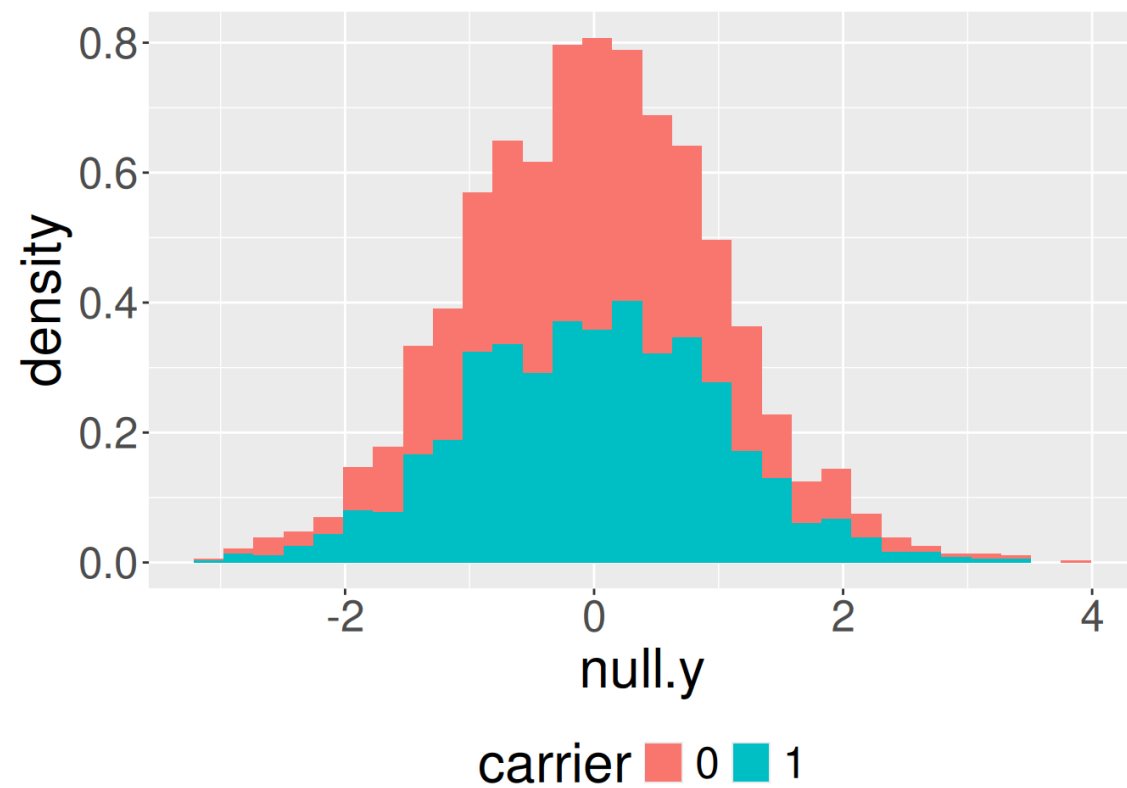
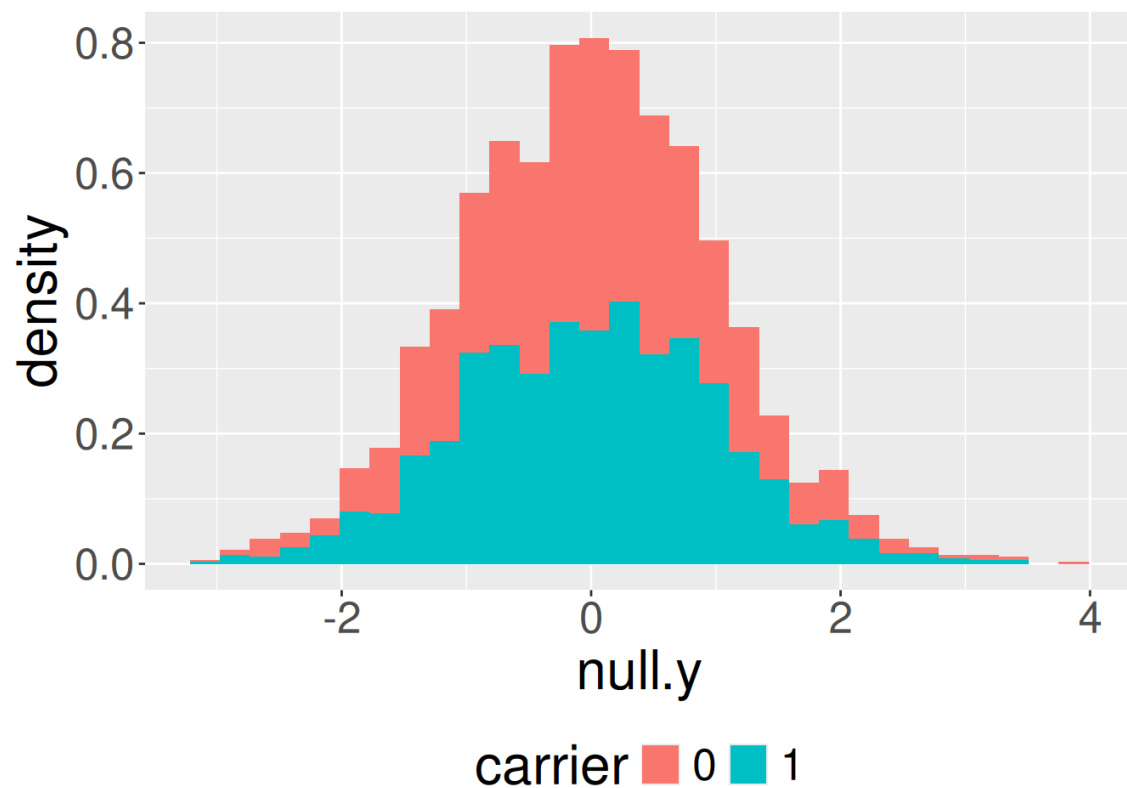
Permute *carrier* group status by *sample()*

```
set.seed(2025)
carrier_permute <- sample(carrier) # permute group labels

## Plot the same null.y phenotype vector vs. permuted labels
ggplot(data.frame(carrier = factor(carrier_permute), null.y = null.y),
       aes(x = null.y, fill = carrier)) +
  geom_histogram(aes(y = stat(density) ))

## Plot the same alt.y phenotype vector vs. permuted labels
ggplot(data.frame(carrier = factor(carrier_permute), alt.y = alt.y),
       aes(x = alt.y, fill = carrier)) +
  geom_histogram(aes(y = stat(density) ))
```

Permute *carrier* group status



Standard two sample t-test results with permuted group labels

```
## Test null.y ~ permuted labels  
t.test(null.y ~ carrier_permute, var.equal = TRUE)
```

```
##  
## Two Sample t-test  
##  
## data: null.y by carrier_permute  
## t = -0.087131, df = 2998, p-value = 0.9306  
## alternative hypothesis: true difference in means between group 0 and group 1 is not equal  
## to 0  
## 95 percent confidence interval:  
## -0.07524942 0.06884617  
## sample estimates:  
## mean in group 0 mean in group 1  
## -0.013094018 -0.009892389
```

Standard two sample t-test results with permuted group labels

```
## Test alt.y ~ permuted labels  
t.test(alt.y ~ carrier_permute, var.equal = TRUE)
```

```
##  
## Two Sample t-test  
##  
## data: alt.y by carrier_permute  
## t = 0.1813, df = 2998, p-value = 0.8561  
## alternative hypothesis: true difference in means between group 0 and group 1 is not equal  
## to 0  
## 95 percent confidence interval:  
## -0.06746119 0.08120784  
## sample estimates:  
## mean in group 0 mean in group 1  
## 0.2288914 0.2220181
```

Permutation test: two-group mean difference

- Consider test statistic that is the mean difference between two groups

$$\mu_{diff} = \widehat{\mu}_1 - \widehat{\mu}_2$$

- Calculate the statistic value for each permuted data set
- Permute group labels for 1000 times

```
# Function to calculate phenotype differences of two groups/vectors
mean.diff <- function(x, y) {
  # x: label vector
  # y: phenotype vector
  x_permuted <- sample(x) # permute x vector
  diff_mean <- mean(y[x_permuted == 1]) - mean(y[x_permuted == 0])
  return(diff_mean) # return the difference of two group means (a.k.a., test statistic value)
}
```

[illegible]

Permutation test: two-group mean difference

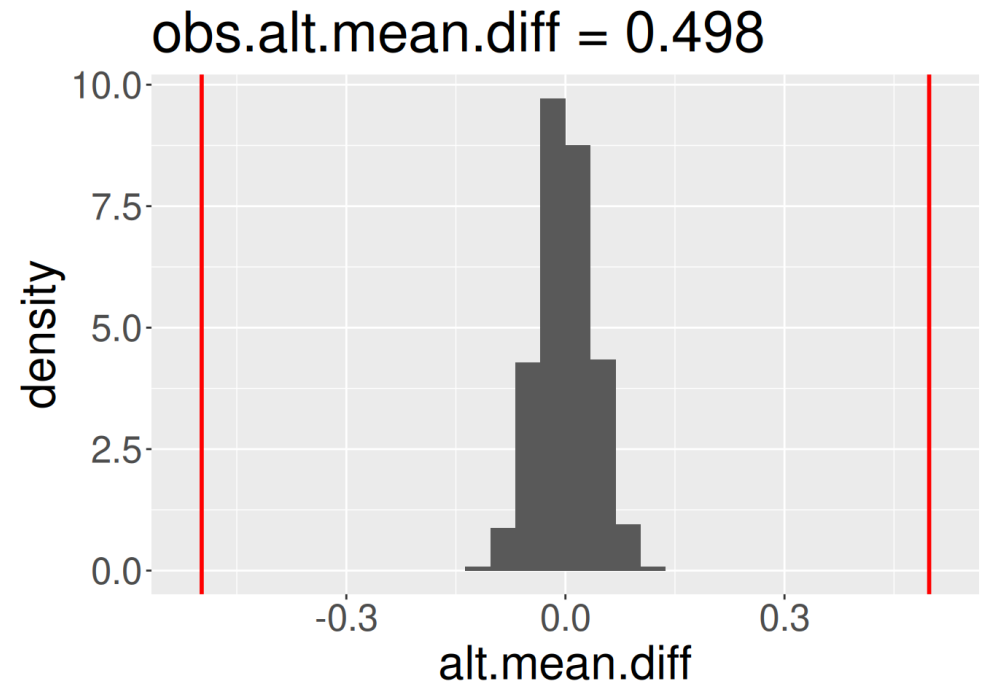
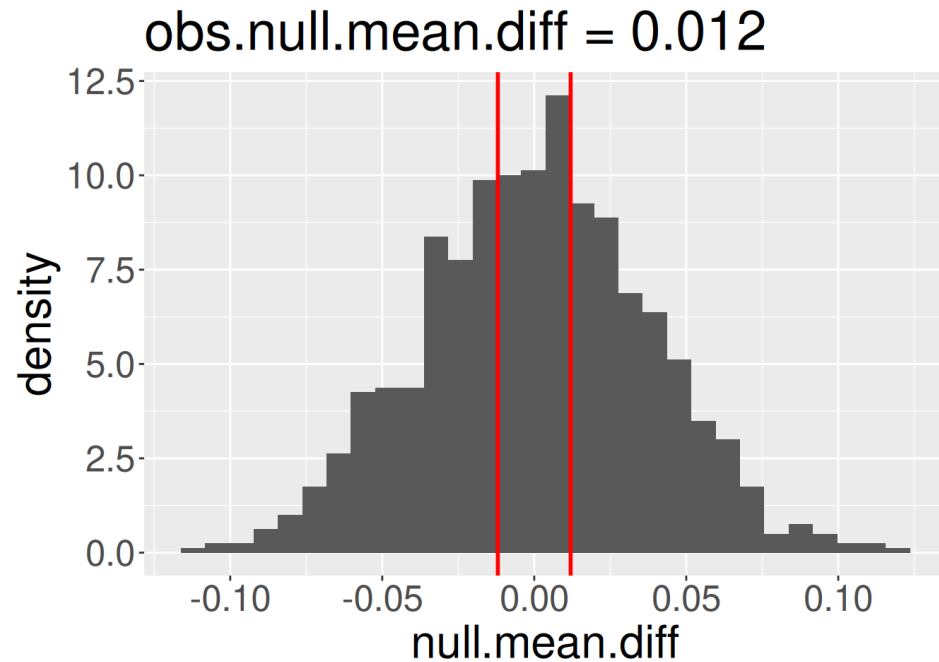
Construct the numeric distribution of μ_{diff}

```
# Observed mean differences
obs.null.mean.diff = mean(null.y[carrier == 1]) - mean(null.y[carrier == 0])
obs.alt.mean.diff = mean(alt.y[carrier == 1]) - mean(alt.y[carrier == 0])

# Histogram of test statistic distributions under the NULL
ggplot(plot_mean_diff, aes(x = null.mean.diff)) +
  geom_histogram(aes(y = stat(density) )) +
  geom_vline(xintercept = c(abs(obs.null.mean.diff), -abs(obs.null.mean.diff)),
            size = 1, col = "red") +
  labs(title = paste("obs.null.mean.diff =", round(abs(obs.null.mean.diff), 3)))
```

Permutation test: two-group mean difference

- Calculate the two-sided p-value from the numeric null distribution



Read line: $\pm$ absolute value of observed mean difference

Permutation test: two-group mean difference

- Calculate the p-value from the numeric null distribution
 - Count the total number of permutations resulting in two-group mean differences that are more extreme than the observed mean difference
 - Then divide by the total number of permutations
 - Proportion of permutations resulting more extreme group mean differences than the observed one

```
# Calculating permutation p-values
sum(abs(null.mean.diff_vec) > abs(obs.null.mean.diff))
```

```
## [1] 747
```

```
sum(abs(null.mean.diff_vec) > abs(obs.null.mean.diff)) / 1000
```

```
## [1] 0.747
```

```
p_null = mean(abs(null.mean.diff_vec) > abs(obs.null.mean.diff))
p_null
```

```
## [1] 0.747
```

```
sum(abs(alt.mean.diff_vec) > abs(obs.alt.mean.diff))
```

```
## [1] 0
```

```
sum(abs(alt.mean.diff_vec) > abs(obs.alt.mean.diff)) / 1000
```

```
## [1] 0
```

```
p_alt = mean(abs(alt.mean.diff_vec) > abs(obs.alt.mean.diff))
p_alt
```

```
## [1] 0
```

Permutation p-value

Now consider a permutation test that randomly permutes the data B times (instead of all $\binom{N}{n}$ times). A permutation test approximates a randomization test. In fact, the permutation test can be analyzed using the following binomial random variable:

$X_P = \#$ permutations out of B that give a more extreme value than the observed test statistic

$$X_P \sim \text{Bin}(p_R, B)$$

$$SE(X_P/B) = \sqrt{\frac{p_R(1-p_R)}{B}} \approx \sqrt{\frac{\hat{p}_P(1-\hat{p}_P)}{B}}$$

Consider a situation where interest is in a small effect, say $p\text{-value} \approx 0.01$. The SE should be less than 0.001.

$$\begin{aligned} 0.001 &= \sqrt{(0.01) \cdot (0.99)/B} \\ B &= (0.01) \cdot (0.99)/(0.001)^2 \\ &= 9900 \end{aligned}$$

Another way to look at the same problem is to use the estimated $p\text{-value} = \hat{p}_P = \frac{X_P}{B}$ to come up with a confidence interval for p_R .

$$\text{CI for } p_R \approx \hat{p}_P \pm 1.96 \sqrt{\frac{\hat{p}_P(1-\hat{p}_P)}{B}}$$

Determine the total of needed permutations to achieve the desired permutation_p-value and $SE(\text{permutation_p-value} = X_p / B)$

How many
permutations are
needed if using
significance
threshold 0.001?



Example test statistics



| Data | Hypothesis Question | Statistic |
|-----------------------------|-----------------------------|--|
| 2 categorical variables | diff in prop | $\hat{p}_1 - \hat{p}_2$ or χ^2 |
| | ratio of prop | $\hat{p}_1/\hat{p}_2$ |
| 1 numeric | diff in means | $\overline{X}_1 - \overline{X}_2$ |
| 1 binary | ratio of means | $\overline{X}_1/\overline{X}_2$ |
| | diff in medians | $\text{median}_1 - \text{median}_2$ |
| | ratio of medians | $\text{median}_1/\text{median}_2$ |
| | diff in SD | $s_1 - s_2$ |
| | diff in var | $s_1^2 - s_2^2$ |
| | ratio of SD or VAR | s_1/s_2 |
| 1 numeric | diff in means | $\sum n_i(\overline{X}_i - \overline{X})^2$ or |
| k groups | | F stat |
| paired or repeated measures | (permute <i>within</i> row) | $\overline{X}_1 - \overline{X}_2$ |
| regression | correlation | least sq slope |
| time series | no serial corr | lag 1 autocross |

Depending on the data, hypotheses, and original data collection structure (e.g., random sampling vs random allocation), the choice of statistic for the permutation test will vary.

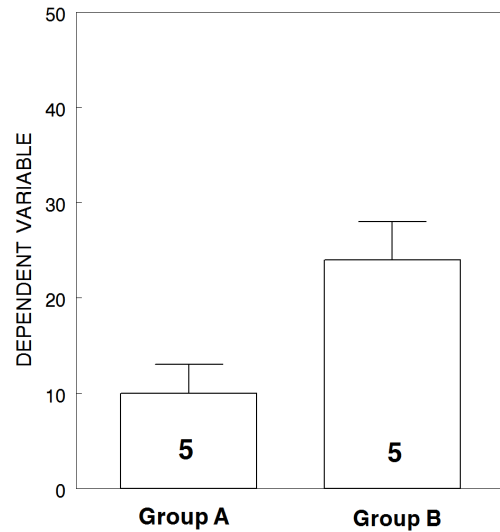
Pros and Cons of Permutation test

| Pros | Cons |
|---|---|
| Require no known test statistic distribution under the null hypothesis | Give the same p-value as parametric tests such as t-test, when data satisfy the assumptions of parametric tests |
| No distribution assumptions for the observed data. Preferred with small sample sizes. | Computationally expensive (could consider a flexible number of permutations per test or parallel computing) |
| Useful when there is no known distribution, aka, analytical pdf/cdf formula, for the test statistic under the null hypothesis | Need to increase the number of permutations according to the significance level |

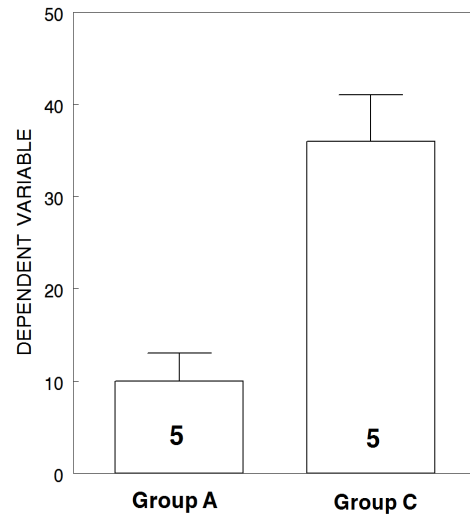
When permutation test is useful

- Suppose we test additive effects of 8 SNPs, one at a time, and we want to know if the most significant association is real.
- For each SNP, the Z-statistic from a logistic regression model has a Normal distribution.
- We need to know the distribution of the most extreme of eight Z-statistics.

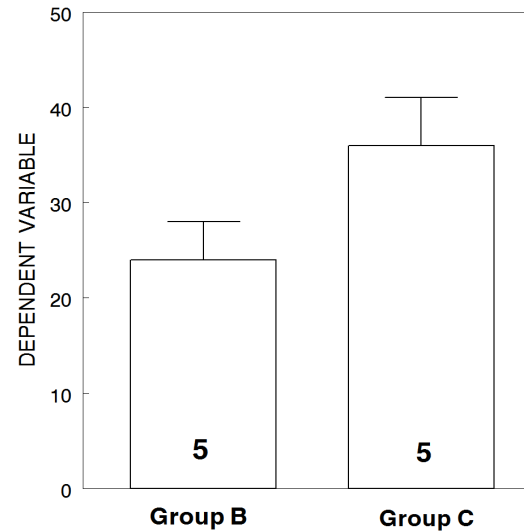
Minimum p-value (or Maximum Z-score Statistic) out of multiple testing can not be directly used to test the hypothesis



$$H_0: \mu_A = \mu_B$$
$$\alpha = 5\%$$



$$H_0: \mu_A = \mu_C$$
$$\alpha = 5\%$$



$$H_0: \mu_B = \mu_C$$
$$\alpha = 5\%$$

If we do multiple pair-wise Two-sample **t-test**

Why Multiple Testing Matters

In general, if we perform m hypothesis tests, what is the probability of at least 1 false positive?

$$P(\text{Making an error}) = \alpha$$

$$P(\text{Not making an error}) = 1 - \alpha$$

$$P(\text{Not making an error in } m \text{ tests}) = (1 - \alpha)^m$$

$$P(\text{Making at least 1 error in } m \text{ tests}) = 1 - (1 - \alpha)^m$$

3 Hypothesis tests

Family-Wise Type 1 Error: $14.2\% = 1 - (1 - 0.05)^3$
(Week 14 Lecture about Multiple Testing)

Permutation for minimum order statistic

- Consider a binary disease phenotype with values “0” for control and values “1” for disease
- Test the association between a gene and the binary disease phenotype
- A total of eight SNPs within the test gene
- Test statistic: **minimum p-value** of single variant tests across all eight SNPs, i.e., **maximum abs(Z-score)** of single variant tests across all eight SNPs
- Analytical distribution for this minimum order statistic is not trivial, as **maximum abs(Z-score)** no longer follows a $N(0, 1)$ distribution under the null

Simulate a binary disease phenotype and genotype data of eight SNPs

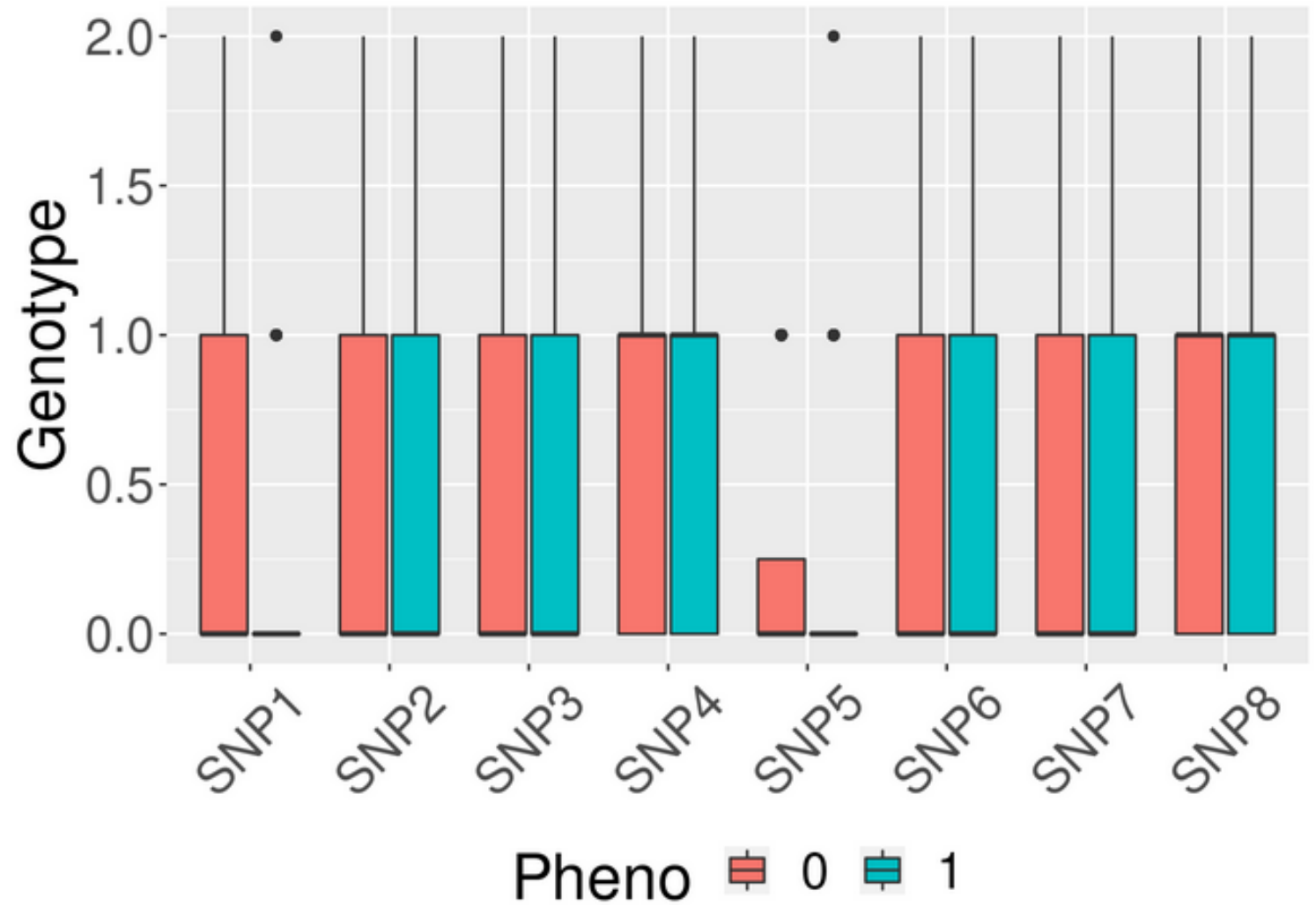
- Simulate 200 samples with 100 controls and 100 cases
- Simulate genotype data of eight SNPs with minor allele frequency (0.1, 0.2, 0.2, 0.4, 0.1, 0.2, 0.2, 0.4)

```
set.seed(2021)
Pheno <- factor(rep(0:1,each=100)) # simulated as independent of SNPs

## simulate genotype data of 8 SNPs
SNP_dat <- data.frame(SNP1=rbinom(200,2,.1),
  SNP2=rbinom(200,2,.2),SNP3=rbinom(200,2,.2),
  SNP4=rbinom(200,2,.4),SNP5=rbinom(200,2,.1),
  SNP6=rbinom(200,2,.2),SNP7=rbinom(200,2,.2),
  SNP8=rbinom(200,2,.4)
)

# Boxplot per SNP per phenotype status
## Melt SNP genotype matrix of 8 columns into 2 columns
SNP_dat_plot <- melt(SNP_dat, value.name = "Genotype", variable.name = "SNP")
SNP_dat_plot$Pheno <- rep(Pheno, 8) # Add phenotype vector
## Boxplot of SNPs ~ phenotypes
ggplot(SNP_dat_plot, aes(x = factor(SNP), y = Genotype,
  fill = factor(Pheno) )) +
  geom_boxplot() + labs(fill = "Pheno", x = NULL) +
  theme(axis.text.x=element_text(angle=45, vjust = 0.5))
```

Simulate a
binary disease
phenotype and
genotype data
of eight SNPs



Is this a null
scenario?



Consider 8 individual tests

- Adjusting for multiple testing by Bonferroni correction
- With significance level $\alpha = 0.05/8 = 0.00625$

```
# Calculate Z-score statistic for one SNP
oneZ <- function(outcome, snp){
  model <- glm(outcome ~ snp, family=binomial())
  Zscore <- coef(summary(model))["snp", "z value"]
  return(Zscore)
}

# ?sapply
## Get Z-score statistic values for all 8 SNPs by sapply
Zscore_8snps <- sapply(SNP_dat, function(snp){oneZ(outcome = Pheno, snp)} ) %>% abs()
Zscore_8snps
```

```
##      SNP1      SNP2      SNP3      SNP4      SNP5      SNP6      SNP7      SNP8
## 3.1394731 0.9885851 0.3653994 0.9268829 0.8216369 0.3681295 1.9663342 0.7882986
```

```
## Get p-values from Z-score statistic values
pvalue_8snps <- 2 * pnorm(Zscore_8snps, lower.tail = FALSE)
pvalue_8snps
```

```
##      SNP1      SNP2      SNP3      SNP4      SNP5      SNP6
## 0.001692519 0.322866168 0.714813255 0.353987332 0.411283597 0.712776656
##      SNP7      SNP8
## 0.049260019 0.430522075
```

```
## Get minimum p-value
min(pvalue_8snps)
```

```
## [1] 0.001692519
```

```
# Adjusting for multiple testing by Bonferroni correction
min(pvalue_8snps) < (0.05 / 8)
```

```
## [1] TRUE
```

Observed minimum p-value

```
# Alternative way of calculating the maximum absolute value of Z-score statistics for all SNPs, or write a loop
```

```
maxZ <- function(outcome, snps){  
  allZs <- sapply(snps, function(snp){oneZ(outcome, snp)} ) %>% abs()  
  return( max(allZs) )  
}
```

```
# Minimum p-value of eight single variant test
```

```
obs.max.Z <- maxZ(outcome = Pheno, snps = SNP_dat)  
p.min <- 2 * pnorm(obs.max.Z, lower.tail = FALSE)  
p.min
```

```
## [1] 0.001692519
```

Permutation for minimum order statistic

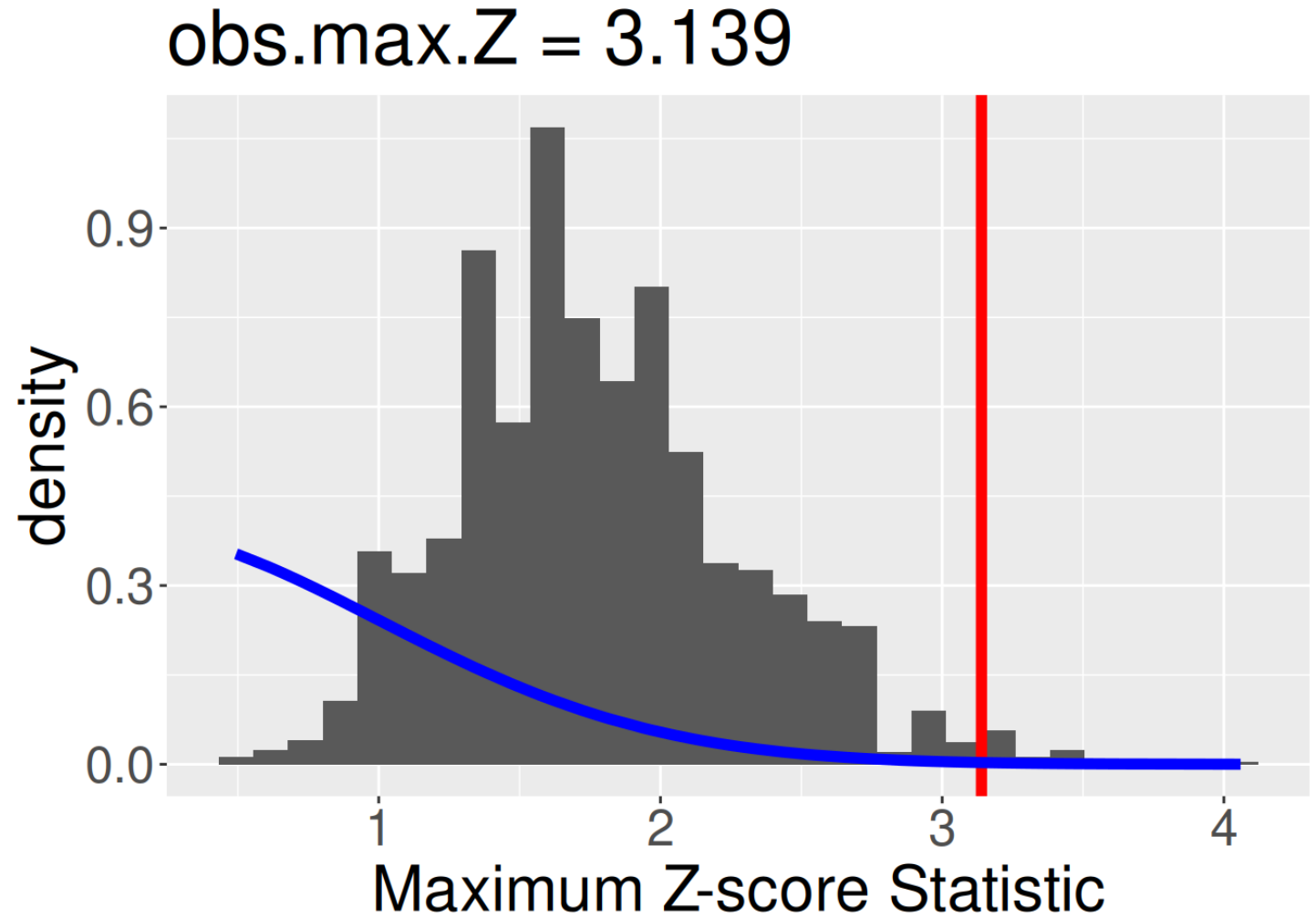
- Generating data sets under null hypothesis by permuting the disease labels
 - No association between the permuted disease labels and SNPs
 - AKA, the permuted disease outcome is independent of all eight SNPs in the gene

```
set.seed(2025)
# ?replicate
many.permZ <- replicate(2000, maxZ(outcome = sample(Pheno), snps = SNP_dat)) # Get max Z-scores based on 1000 permuted phenotype labels
permZ_plot <- data.frame(permZ = many.permZ, normal_density = dnorm(many.permZ))

ggplot(permZ_plot, aes(x = permZ)) +
  geom_histogram(aes(y = stat(density) )) +
  geom_vline(xintercept = c(abs(obs.max.Z)),
             size = 2, col = "red") +
  geom_line(aes(x = many.permZ, y = dnorm(many.permZ)), size = 2, col = "blue") +
  labs(title = paste("obs.max.Z =", round(abs(obs.max.Z), 3)),
       x = "Maximum Z-score Statistic")
```

Distribution of permuted maximum Z-score statistic

- Blue line: $N(0, 1)$ density function
- Red line: Observed maximum Z-score statistic value



Permutation test p-value for the maximum Z-score statistic

```
# Permutation test p-value  
p.min.perm <- mean( many.permZ > obs.max.Z )  
p.min.perm
```

```
## [1] 0.011
```

When you get
permutation p-value < 0.05 for a set of data
generated under the
null hypothesis!

What dose this mean?



Complete In-Class
Activity, submit the
knitted html file with
your answers to canvas
for Participation Credits



R package **coin** (conditional inference)

- The **coin** package implements a unified approach to **permutation tests** providing a huge class of independence tests for nominal, ordered, numeric, and censored data as well as multivariate data at mixed scales.
- Provide conditional versions (permutation tests) of classical tests, such as tests for location and scale problems in two or more samples, independence in two- or three-way contingency tables, or association problems for censored, ordered categorical or multivariate data.
- Approximations of the exact null distribution via the **limiting distribution** (**`asymptotic`**) or **conditional Monte Carlo resampling** (**`approximate`**) are available for every test procedure.
- The **exact** null distribution is currently available for univariate two-sample problems only.

R library *coin*

- Framework was developed by Strasser and Weber (1999): theoretical insights of a unified treatment of a huge class of permutation tests
- Generic functions for obtaining statistics, conditional expectation and covariance matrices as well as p-value, distribution, density and quantile functions for the reference distribution
 - Help to extract information from these objects
 - Conveniently interfaced in the function ***independence_test()***

General Independence Test

Description

Testing the independence of two sets of variables measured on arbitrary scales.

Usage

```
## S3 method for class 'formula'
independence_test(formula, data, subset = NULL, weights = NULL, ...)
## S3 method for class 'table'
independence_test(object, ...)
## S3 method for class 'IndependenceProblem'
independence_test(object, teststat = c("maximum", "quadratic", "scalar"),
                  distribution = c("asymptotic", "approximate",
                                   "exact", "none"),
                  alternative = c("two.sided", "less", "greater"),
                  xtrafo = trafo, ytrafo = trafo, scores = NULL,
                  check = NULL, ...)
```

Apply `independence_test()` to Example 1

```
independence_test(null.y ~ carrier, data = null_dt,  
                  ytrafo = rank_trafo, distribution = "asymptotic") # coin package
```

```
##  
## Asymptotic General Independence Test  
##  
## data: null.y by carrier (0, 1)  
## Z = -0.077442, p-value = 0.9383  
## alternative hypothesis: two.sided
```

```
independence_test(alt.y ~ carrier, data = alt_dt,  
                  ytrafo = rank_trafo, distribution = "asymptotic") # coin package
```

```
##  
## Asymptotic General Independence Test  
##  
## data: alt.y by carrier (0, 1)  
## Z = -12.056, p-value < 2.2e-16  
## alternative hypothesis: two.sided
```

How dose *independence_test()* work?

- The data are pre-processed along with their transformations
- Deviations from independence are captured by a (possibly multivariate) linear statistic
- Standardized by conditional expectation and variance, and aggregated to a final test statistic (scalar, maximum, or quadratic)
- Consider testing the independence between two variables Y and X, under a certain block structure of the observations (B) — for example, study centers in a multi-center randomized clinical trial where only a re-randomization of observations within blocks is admissible.

$$H_0 : D(Y|X, B) = D(Y|B)$$

- Strasser and Weber (1999) suggest deriving scalar test statistics for testing H_0 from multivariate linear statistics of the form

$$\mathbf{T} = \sum_{j=1}^k \mathbf{T}_j \in \mathbb{R}^{pq}$$

How dose *independence_test()* work?

- The linear statistic for each block is given by

$$\mathbf{T}_j = \text{vec} \left(\sum_{i=1}^n I(b_i = j) w_i g(\mathbf{X}_i) h(\mathbf{Y}_i)^\top \right) \in \mathbb{R}^{pq}.$$

The function $I(\cdot)$ is the indicator function and vec denotes the vec operator (which stacks the columns of a matrix). Here, $g : \mathcal{X} \rightarrow \mathbb{R}^{p \times 1}$ is a transformation of the $\mathbf{X}$ measurements and $h : \mathcal{Y} \rightarrow \mathbb{R}^{q \times 1}$ is a transformation of the $\mathbf{Y}$ values. The function $h(\mathbf{Y}_i) = h(\mathbf{Y}_i, (\mathbf{Y}_1, \dots, \mathbf{Y}_n))$ is also called *influence function* and may depend on the full vector of responses $(\mathbf{Y}_1, \dots, \mathbf{Y}_n)$, however only in a permutation symmetric way, i.e., the value of the function must not depend on the order in which $\mathbf{Y}_1, \dots, \mathbf{Y}_n$ appear. The case weights w_i are assumed to be integer-valued, indicating that w_i observations with realizations $\mathbf{Y}_i$, $\mathbf{X}_i$ and b_i are available, with default $w_i \equiv 1$.

How dose *independence_test()* work?

The distribution of $\mathbf{T}$ depends on the joint distribution of $\mathbf{Y}$ and $\mathbf{X}$, which is unknown under almost all practical circumstances. At least under the null hypothesis one can dispose of this dependency by fixing $\mathbf{X}_1, \dots, \mathbf{X}_n$ and conditioning on all possible permutations of the responses $\mathbf{Y}_1, \dots, \mathbf{Y}_n$ within block $j, j = 1, \dots, k$. The conditional expectation $\mu \in \mathbb{R}^{pq}$ and covariance $\Sigma \in \mathbb{R}^{pq \times pq}$ of $\mathbf{T}$ under H_0 given all permutations $\sigma \in S$ of the responses are derived by [Strasser and Weber \(1999\)](#) and are given in [Appendix A](#). Having the conditional expectation and covariance at hand we are able to standardize an observed linear statistic $\mathbf{t} \in \mathbb{R}^{pq}$ (of the form given in [Equation 1](#)) and aggregate it to some univariate test statistic $c = c(\mathbf{t}, \mu, \Sigma)$. Various choices for $c(\mathbf{t}, \mu, \Sigma)$ are conceivable, e.g., a quadratic form or a maximum type statistic (see [Section 3](#)). In the latter case, a natural first step is to standardize each of the pq statistics in $\mathbf{t}$ by its expectation and standard deviation:

$$\mathbf{z} = \text{diag}(\Sigma)^{-1/2}(\mathbf{t} - \mu). \quad (2)$$

μ = Conditional expectation of $\mathbf{T}$ under Null

Σ = Conditional variance-covariance of $\mathbf{T}$ under Null

Hothorn2008, JSS.

Data Structure

We are provided with n observations $(\mathbf{Y}_i, \mathbf{X}_i, b_i, w_i), i = 1, \dots, n$. In addition to variables $\mathbf{X}$, $\mathbf{Y}$, and b , it is convenient (for example to efficiently represent large contingency tables) to include case weights w_i , defaulting to $w_i \equiv 1$. This data structure is represented by class `'IndependenceProblem'`:

Class `'IndependenceProblem'`

| Slot | Class |
|----------------------|---------------------------|
| <code>x</code> | <code>'data.frame'</code> |
| <code>y</code> | <code>'data.frame'</code> |
| <code>block</code> | <code>'factor'</code> |
| <code>weights</code> | <code>'numeric'</code> |

Note that objects of this class implicitly define the null distribution H_0 and all admissible permutations of observations within blocks.

Independence Problems and Linear Statistics

The transformation functions g and h as well as the transformed observations $g(\mathbf{X}_i)$ and $h(\mathbf{Y}_i)$, $i = 1, \dots, n$, are added to the data structure by extending class 'IndependenceProblem':

Class 'IndependenceTestProblem'
Contains 'IndependenceProblem'

| Slot | Class |
|--------|------------|
| xtrans | 'matrix' |
| ytrans | 'matrix' |
| xtrafo | 'function' |
| ytrafo | 'function' |

The `ytrafo` and `xtrafo` slots correspond to the transformations h and g , respectively. The i th row of the $n \times q$ matrix `ytrans` corresponds to $h(\mathbf{Y}_i)$. Similarly, the rows of `xtrans` ($n \times p$) correspond to $g(\mathbf{X}_i)$. Note that, in addition to the data, hypothesis and permutation scheme, the test statistic $\mathbf{T}$ is defined by objects of class 'IndependenceTestProblem' as well.

In the simplest case of both $\mathbf{X}$ and $\mathbf{Y}$ being univariate factors at p and q levels, g and h are the corresponding dummy codings and the linear statistic $\mathbf{T}$ is the (vectorized) $p \times q$ contingency table of $\mathbf{X}$ and $\mathbf{Y}$. In the rats example, the default dummy coding for factor `group` is employed and a rank transformation (via `rank()`) is applied to `time`.

Derive conditional null distribution

| Class | Description |
|--------------------------|---|
| "ExactNullDistribution" | Exact conditional null distribution (e.g., computed via the shift algorithm). |
| "ApproxNullDistribution" | Approximation of the exact conditional distribution using conditional Monte Carlo procedures. |
| "AsymptNullDistribution" | Asymptotic conditional distribution (via multivariate normal or χ^2 distribution). |

For the most important special cases, suitable function *generators* are provided in **coin**. For example, the function `approximate(nresample = 1000)` returns a Monte Carlo function that draws `nresample` (default: 10000) random permutations. Similarly, `exact()` and `approximate()` return functions computing the exact or asymptotic null distributions, respectively. Again, computational details in the computation of the null distribution can be controlled via arguments of the function generators.

independence_test()

```
independence_test(object,  
  teststat = c("max", "quad", "scalar"),  
  distribution = c("asymptotic", "approximate", "exact"),  
  alternative = c("two.sided", "less", "greater"),  
  xtrafo = trafo, ytrafo = trafo, scores = NULL,  
  check = NULL, ...)
```

Thus, `object` describes the data and the null hypothesis. Arguments `xtrafo` and `ytrafo` refer to the transformations g and h : Both are by default set to function `trafo()` which chooses suitable transformations based on the scale of the considered variables (see below). The three types of univariate test statistics discussed above are hard-coded and can be specified by a simple string. Similarly, the reference distribution and the alternative hypothesis can be supplied as strings. In addition to these simple specifications, more flexible specifications for the data (`object`), the transformations (`xtrafo`, `ytrafo`), and the distribution are available, as discussed in the following. The `scores` argument takes a named list of numeric vectors to be used as scores for ordered factors. Validity checks for objects of class ‘`IndependenceProblem`’ specified to argument `check` can be used to test for certain aspects of the data, e.g., when one has to make sure that two independent samples are present.

R library *coin*

| Test | xtrafo <i>g</i> | ytrafo <i>h</i> | teststat <i>c</i> |
|---------------------------------------|-----------------|-----------------|-------------------|
| Independent samples | | | |
| Wilcoxon-Mann-Whitney | f_trafo() | rank() | "scalar" |
| Normal quantiles | f_trafo() | normal_trafo() | "scalar" |
| Median | f_trafo() | median_trafo() | "scalar" |
| Ansari-Bradley | f_trafo() | ansari_trafo() | "scalar" |
| Log rank | f_trafo() | logrank_trafo() | "quad" |
| Kruskal-Wallis | f_trafo() | rank() | "quad" |
| Fligner | f_trafo() | fligner_trafo() | "quad" |
| Spearman | rank() | rank() | "scalar" |
| Cochran-Mantel-Haenszel | f_trafo() | f_trafo() | "quad" |
| Pearson's χ^2 | f_trafo() | f_trafo() | "quad" |
| Cochran-Armitage / Linear Association | scores | any | "scalar" |
| <i>K</i> -sample permutation test | f_trafo() | any | any |
| Maximally-selected statistics | maxstat_trafo() | any | "max" |
| Dependent samples | | | |
| Friedman | f_trafo() | rank() | "quad" |
| Maxwell-Stuart | f_trafo() | f_trafo() | "quad" |
| Wilcoxon signed rank | f_trafo() | rank() | "scalar" |

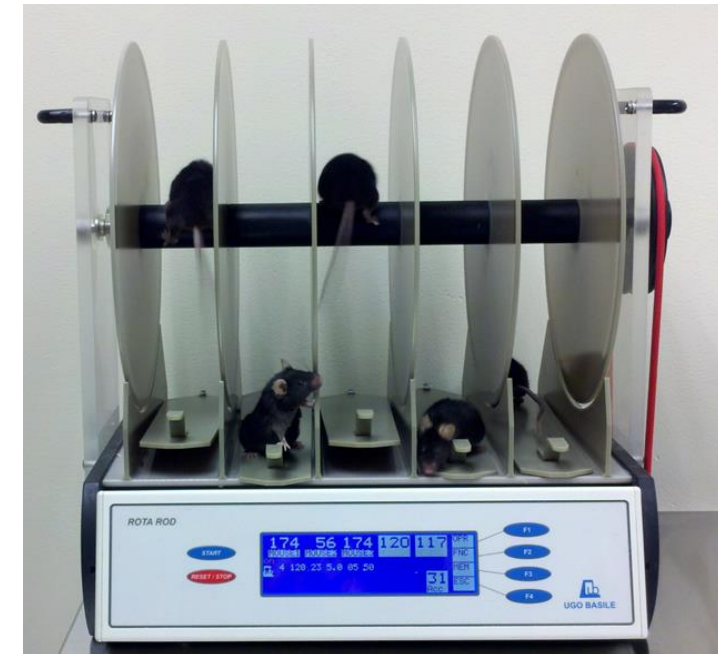
Table 4: Representations of the conditional counterparts of important classical tests in **coin**.

Example: two-sample test (location)

| group | time | | | | | | | | | | | |
|-----------|------|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|
| control | 300 | 300 | 300 | 300 | 300 | 300 | 300 | 300 | 300 | 300 | 300 | 300 |
| treatment | 18 | 22 | 75 | 163 | 271 | 300 | 300 | 300 | 300 | 300 | 300 | 300 |

Table 1: The rotarod data: length of time on rotating cylinder by group.

- $n = 24$ rats received a fixed oral dose of a centrally acting muscle relaxant as active treatment or a saline solvent as control.
- The animals were placed on a rotating cylinder and the length of time each rat remained on the cylinder was measured, up to a maximum of 300 seconds.
- The rats were randomly assigned to the control and treatment groups.
- A permutation test is the appropriate way to investigate if the response is independent of the group assignment.



Compare with *t.test()* from *stats* package

```
t.test(time ~ group, data = rotarod) # stats package
```

```
##  
## Welch Two Sample t-test  
##  
## data: time by group  
## t = 2.3379, df = 11, p-value = 0.03932  
## alternative hypothesis: true difference in means between group control and group treatment  
## is not equal to 0  
## 95 percent confidence interval:  
## 4.641392 153.858608  
## sample estimates:  
## mean in group control mean in group treatment  
## 300.00 220.75
```

t.test() is not appropriate for this dataset.

Example: two-sample test (location)

The data are particularly challenging because of the many ties in the (right-censored) response (19 observations take the maximal value 300) and the quasi-complete separation (smaller values of time are only observed in the treatment group).

```
# Load example data set
data("rotarod", package = "coin")

# Call the interface function
independence_test(time ~ group, data = rotarod,
  ytrafo = rank_trafo, distribution = "exact") # coin package

##
##  Exact General Independence Test
##
## data:  time by group (control, treatment)
## Z = 2.4389, p-value = 0.03727
## alternative hypothesis: two.sided
```

Here, the conditional **Wilcoxon-Mann-Whitney test** was performed via a **rank transformation of the response**, employing the **exact distribution** for obtaining the p-value.

Example: two-sample test (location)

- Input argument of *distribution = "asymptotic"* denotes the conditional null distribution of the test statistic can be approximated by its asymptotic distribution (default).

```
independence_test(time ~ group, data = rotarod,  
  ytrafo = rank_trafo, distribution = "asymptotic") # coin package
```

```
##  
## Asymptotic General Independence Test  
##  
## data:  time by group (control, treatment)  
## Z = 2.4389, p-value = 0.01473  
## alternative hypothesis: two.sided
```

Example: two-sample test (location)

- Conditional null distribution of the test statistic can also be approximated via Monte Carlo resampling (*distribution = "approximate"*).

```
independence_test(time ~ group, data = rotarod,  
  ytrafo = rank_trafo, distribution = "approximate") # coin package
```

```
##  
##  Approximative General Independence Test  
##  
## data:  time by group (control, treatment)  
## Z = 2.4389, p-value = 0.0382  
## alternative hypothesis: two.sided
```

Compare with *wilcox.test()* from *stats* package



```
wilcox.test(time ~ group, data = rotarod, exact = TRUE, paired = FALSE) # stats package
```

```
##  
## Wilcoxon rank sum test with continuity correction  
##  
## data: time by group  
## W = 102, p-value = 0.01647  
## alternative hypothesis: true location shift is not equal to 0
```

Apply
``independence_test()``
to Example 2

```
temp = independence_test(Pheno ~ SNP1 + SNP2 + SNP3 + SNP4 + SNP5 +  
                        SNP6 + SNP7 + SNP8,  
                        data = data.frame(Pheno, SNP_dat),  
                        teststat = "maximum",  
                        distribution = "asymptotic") # coin package
```

```
temp
```

```
##  
## Asymptotic General Independence Test  
##  
## data: Pheno by  
## SNP1, SNP2, SNP3, SNP4, SNP5, SNP6, SNP7, SNP8  
## maxT = 3.2718, p-value = 0.008506  
## alternative hypothesis: two.sided
```

R library *coin*

- Framework was developed by Strasser and Weber (1999)
 - Theoretical insights of a unified treatment of a huge class of permutation tests
- Salient parts of the Strasser-Weber framework are elucidated by Hothorn et al. (2006)
 - Introduce the package and illustrate the transition from theory to practice
- A thorough description of the software implementation is given by Hothorn et al. (2008)

References

- Strasser, H. and Weber, C. (1999). On the asymptotic theory of permutation statistics. *Mathematical Methods of Statistics* 8(2), 220–250.
- Hothorn, T., Hornik, K., van de Wiel, M. A. and Zeileis, A. (2006). A Lego system for conditional inference. *The American Statistician* 60(3), 257–263. doi: 10.1198/000313006X118430
- Hothorn, T., Hornik, K., van de Wiel, M. A. and Zeileis, A. (2008). Implementing a class of permutation tests: The coin package. *Journal of Statistical Software* 28(8), 1–23. doi: 10.18637/jss.v028.i08

Next Lectures

- Homework 7 due 11/13 (Thursday), revision due 11/20 (Thursday)
- Week 13 (11/20): Machine learning (Jingjing)
 - Homework 8 distributed and due 12/04 (Thursday)
- Week 14 (12/01): Adjust for multiple tests (Karen)
- Week 15 (12/08): Rigor and reproducibility (Karen)
 - Homework 8 revision due 12/12 (Friday)